DEPSTAR Department of Computer Engineering

Charotar University of Science and Technology

Devang Patel Institute of Advance Technology and Research

Department of Computer Engineering

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

➔Practical Aim

To implement linear and binary search using both iterative and recursive approaches.

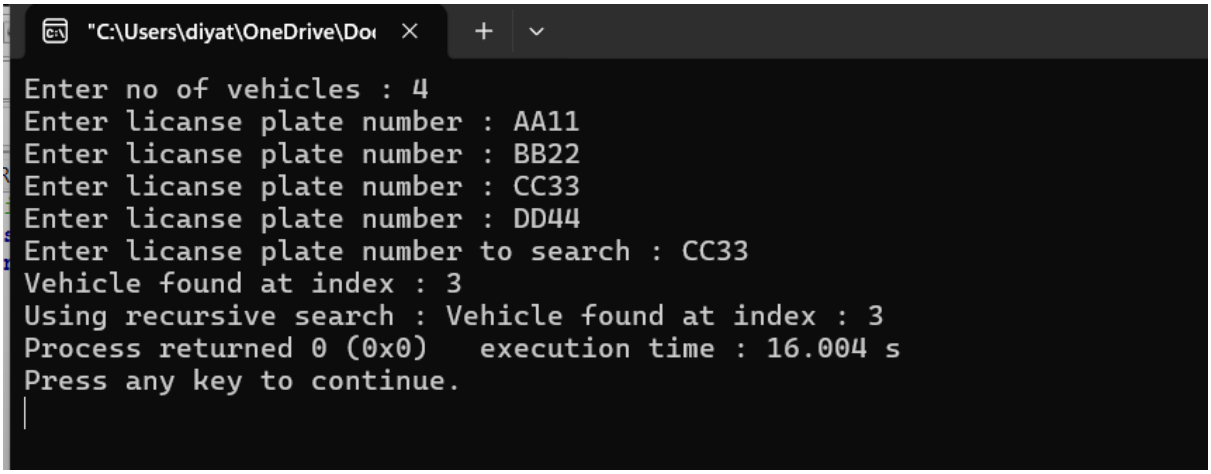
Problem 1

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found. Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Source Code:

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/6a3c760dc37cc4f828716fd7d2fc47aebdfcbbae/PRACTICAL%202

OUTPUT



The program displays the position of the target element found using iterative and recursive linear search. If the element is not present, it displays “Element not found.”

Conclusion:
Thus, linear search was successfully implemented using both iterative and recursive approaches. The target element was searched sequentially, and the position of the target was obtained if it existed in the list.

Problem 2

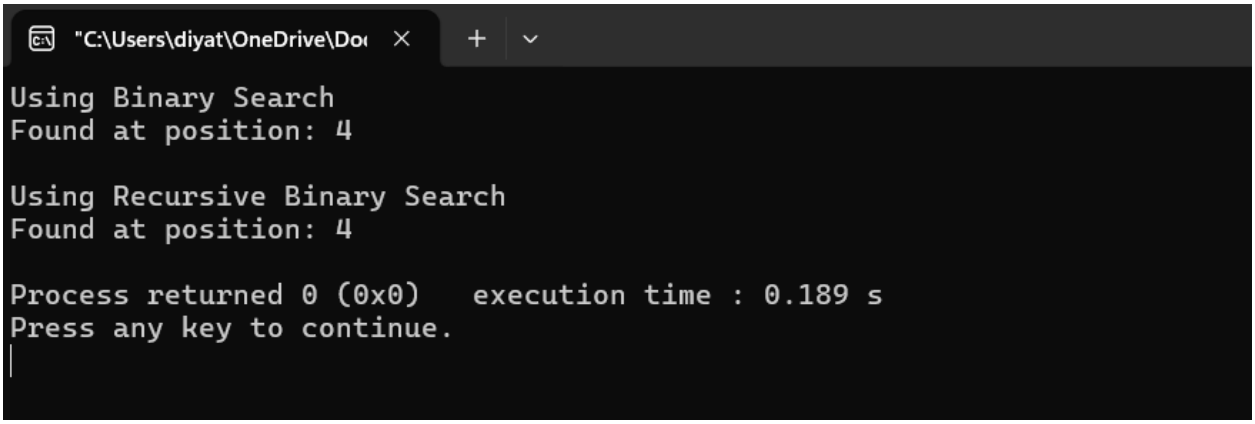
A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

Source CODE

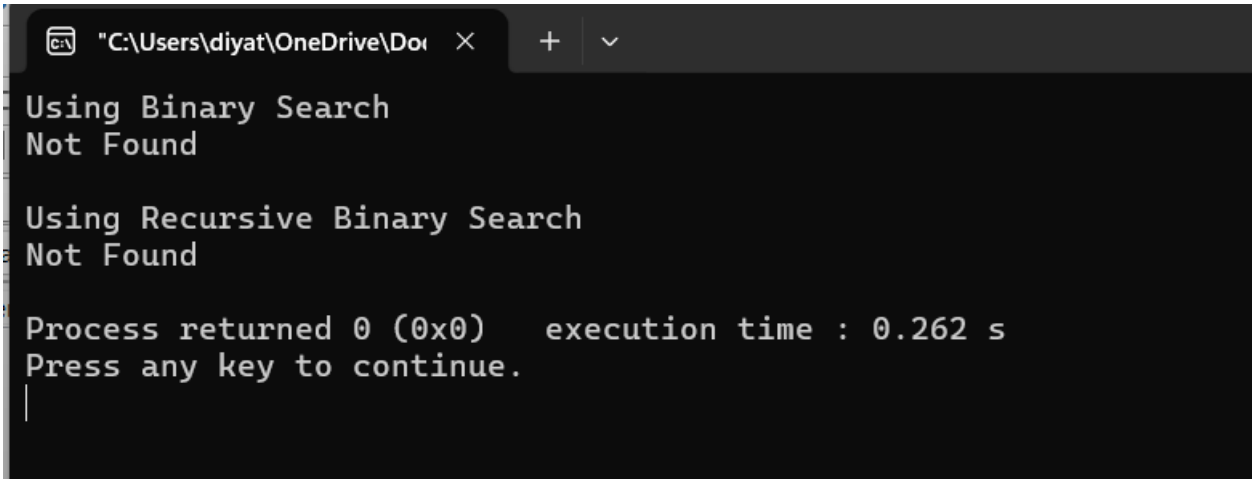
https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/6a3c760dc37cc4f828716fd7d2fc47aebdfcbbae/PRACTICAL%202

OUTPUT

For target 104,



For target 111,



The program displays the position of the target element found using iterative and recursive binary search. Since binary search works on a sorted array, it quickly finds the target. If the element is not present, it displays "Element not found."

Conclusion:

Thus, binary search was successfully implemented using both iterative and recursive approaches. The target element was efficiently located by repeatedly dividing the sorted list into smaller parts.

Challenges Faced :

Problem 1 – Linear Search

Problem Faced:
I was confused about how to implement the recursive approach and where to stop the recursion. I also had difficulty handling the case when the target element was not present.

Solution:
I used a base condition to stop the recursion when the element was found or when the end of the array was reached. I also returned `-1` when the target was not found.

Problem 2 – Binary Search

Problem Faced:
I had difficulty understanding how to update low, high, and mid correctly. I also sometimes got incorrect results when the array was not sorted.

Solution:
I first made sure that the array was sorted. Then I compared the target with the middle element and updated low or high accordingly. I also used a proper base condition in the recursive version.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

Answer: If the target plate is near the end, linear search may need to check almost every element. To skip checking some elements, the input must have some useful property, such as being sorted. Problem 2 shows that a sorted list allows binary search to eliminate half of the elements at each step.

2. [Problem 2] What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

Answer: Binary search requires the input list to be sorted. If the list is not sorted, the decision to search on the left or right side becomes invalid, so the algorithm may return a wrong result or fail to find an element that is actually present.